

4: I/O & Interrupts

TUTORIUM 24 - BuS 2024

Frage 1:

Interrupts vs Polling:

In welchen Fällen ergibt es Sinn, **Polling** zu benutzen?

Frage 1: Wann nutzen wir Polling?

- Polling:
 - bei **kurzen** Events, die **oft** auftreten
 - bei vorhersehbaren Events können wir einen Polling-Loop bauen, um den Aufwand von Interrupts zu umgehen
 - Falls das Gerät keine Interrupts erlaubt oder keine Interrupts mehr frei sind
- Beispiel für Polling: Netzwerkgebundenes Programm
 - Vorhersehbar, wann neue Pakete ankommen
 - Synchronisation und Kontextwechsel von Interrupts werden umgangen

Frage 1: Wann nutzen wir Interrupts?

- Interrupts:
 - Effizienz (keine Verschwendung von CPU-Zyklen durch busy waiting)
 - schnelle Reaktionszeiten
 - asynchrone Ereignisbehandlung: Events treten unvorhersehbar auf
- Beispiel für Interrupts:
 - siehe nächste Aufgabe

Frage 2:

Gegeben sind folgende I/O-Geräte:

- **Maus**
- **Festplatte** mit Benutzerdaten
- **Lichtsensord** (Helligkeitserkennung) am Smartphone

Entscheidet jeweils, ob man diese Geräte mit Polling oder mit Interrupts betreiben sollte.

Bei welchen Geräten würde die Nutzung eines I/O-Puffers Sinn ergeben?

Frage 2: Nutzung von Polling oder Interrupts? Mit Buffer oder ohne?

- Maus - **Interrupts**
 - Mausbewegungen sind sehr unregelmäßig, aber müssen dringend behandelt werden
 - Puffer, damit Bewegungen festgehalten werden können, während wichtigere Operationen ausgeführt werden
- Festplatte – **Interrupts**
 - Daten werden ebenfalls unregelmäßig gelesen/geschrieben
 - Puffer werden eingesetzt, um die Daten auf dem Weg von oder zu der Festplatte zwischenspeichern
- Lichtsensor am Smartphone – **Polling**
 - Kann man periodisch überprüfen, z.B. einmal pro Sekunde
 - Helligkeit des Bildschirms ist nicht zeitkritisch, höhere Latenzen sind nicht schlimm

Frage 3:

Angenommen ein Computer kann in 5 Nanosekunden aus dem Speicher lesen bzw. in den Speicher schreiben.

Wenn ein Interrupt auftritt, werden alle 32 CPU-Register und zusätzlich noch der Program Counter und Stack Pointer auf den Stack gelegt.

Wie viele Interrupts kann dieser Computer maximal pro Sekunde verarbeiten?

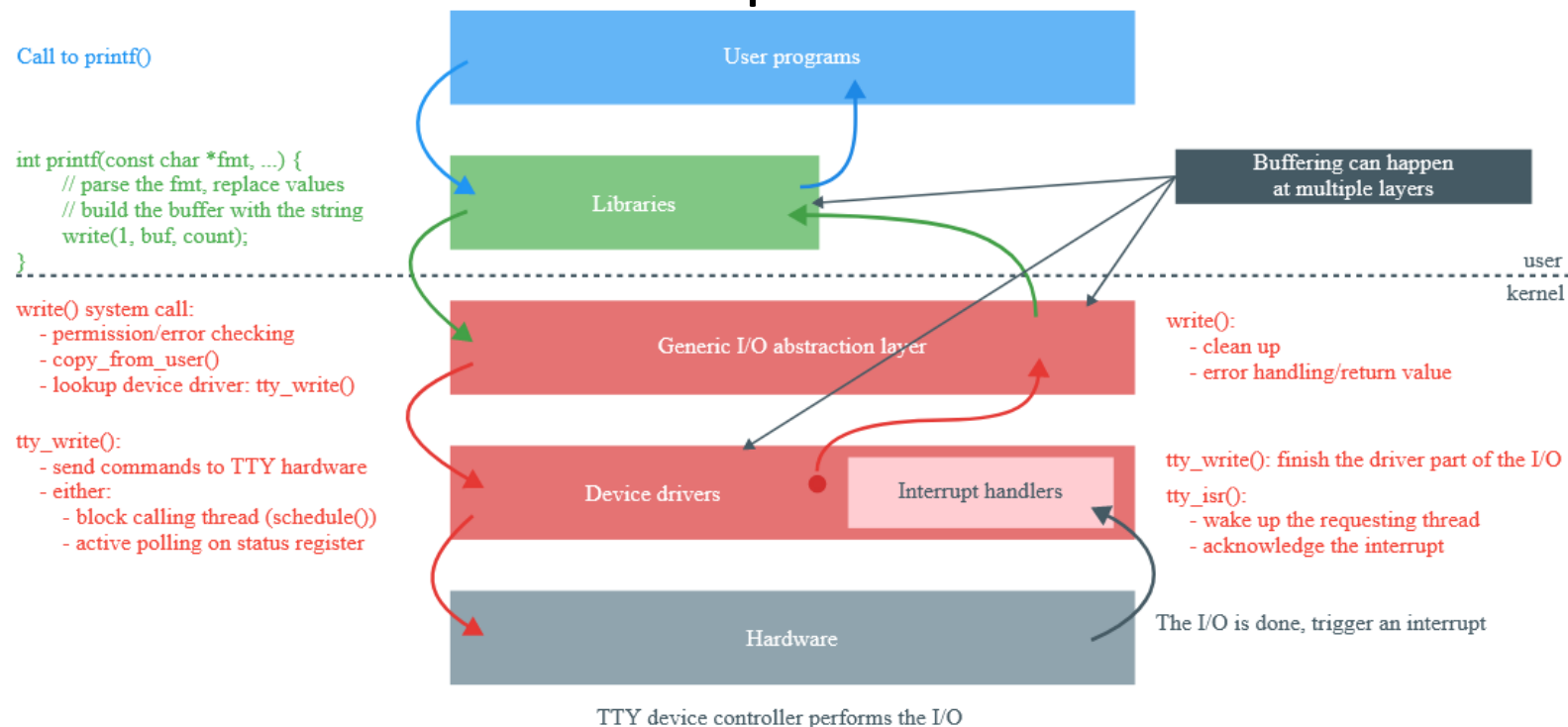
Frage 3: Wie viele Interrupts kann es maximal geben?

- 32 + 2 Register müssen gesichert werden, mit je 5 Nanosekunden
 - $(32 + 2) * 5 \text{ ns} = 170 \text{ ns}$
- Register müssen am der ISR wieder aus dem Speicher gelesen werden
 - $170 \text{ ns} * 2 = 340 \text{ ns}$ insgesamt fürs Speichern und Laden der Register
- Maximale Zahl an Interrupts:
 - $1 / 340\text{e-}9 = \text{ca. } 2.941.176$
also ca. 3 Millionen Interrupts pro Sekunde

Frage 4:

Ordnet den Aufgaben die passende I/O-Schicht zu.

- Berechnen von Track, Sector und Head für einen Festplattenlesevorgang
- Schreiben von Commands in die Register der Geräte
- Überprüfung der User-Berechtigung bei Zugriff auf ein Gerät
- Binärzahlen zu deren ASCII-Repräsentation konvertieren



Frage 4: Aufgaben den I/O Schichten zuordnen

- Berechnen von Track, Sector und Head für einen Festplattenlesevorgang
 - Gerätetreiber/Festplattencontroller
 - Logische Blockadresse muss in physische Blockadresse umgerechnet werden
 - wird von manchen Diskcontroller übernommen, ansonsten muss der Gerätetreiber das übernehmen
- Schreiben von Commands in die Register der Geräte
 - Gerätetreiber
 - Low-Level-Operation, die direkte Interaktion mit der Hardware erfordert
- Überprüfung der User-Berechtigung bei Zugriff auf ein Gerät
 - Generic I/O layer
 - Zugriffskontrolle geschieht auf Kernelebene unabhängig von den einzelnen Geräten
- Binärzahlen zu deren ASCII-Repräsentation konvertieren
 - Gerätetreiber
 - Typische Funktion dieser Schicht, standardisiert Datenrepräsentation unabhängig vom Gerät

Frage 5:

Der Clock Interrupt Handler auf unserem Computer braucht **2 ms pro Clock tick** (Kontextwechsel schon einberechnet).

Die Clock tickt mit **60 Hz**.

Wie viel Prozent der CPU ist für die Clock zuständig?

$60 \text{ Hz} \hat{=} 60 \text{ clock ticks pro s}$

$120 \text{ ms pro Sekunde}$
 $0,12 \text{ s} \rightarrow 12\%$

Frage 5: Wie viel Prozent der CPU ist für die Clock zuständig?

- 2 Millisekunden pro Clock tick
- 60 Hz entsprechen 60 Clock ticks pro Sekunde
 - pro Sekunde werden also $60 * 2 \text{ ms} = 120 \text{ ms}$ für Clock ticks benötigt
- 120 ms entsprechen $120 / 1000 = 0,12$ Sekunden
- Es sind also 12 % der CPU mit der Clock beschäftigt

Frage 6:

Unser Computer nutzt eine programmierbare Clock im Square-Wave-Mode. Es ist ein 500 MHz Oszillator verbaut.

Was sollte der Wert vom zugehörigen Register sein, um eine Clockauflösung von...

- ... einer Millisekunde (also ein Clocktick pro ms)
- ... 100 Mikrosekunden

zu erreichen?

Frage 6: Wert des Registers für die Clockauflösung

- 500 MHz entsprechen 500.000.000 Hz
- Für 1 Millisekunde:
 - 1 ms entsprechen 0,001 Sekunden
 - $0,001 * 500.000.000 = 500.000$
- Für 100 Mikrosekunden:
 - 100 Mikrosekunden entsprechen 0,0001 Sekunden
 - $0,0001 * 500.000.000 = 50.000$
- Wegen des Square-Wave-Modes müssten die Register also nach jedem Interrupt je auf 500.000 bzw. 50.000 zurückgesetzt werden

Frage 7:

Angenommen es dauert 2 Nanosekunden, um ein Byte zu kopieren.

Wie viel Zeit dauert es, auf einem 1920 x 1080 Pixel großen Monitor mit 24-bit Farben ein komplett neues Bild darzustellen? ✓

Wie viele Kopien können wir pro Sekunde machen?

Wie viel Puffer brauchen wir bei einem 60 Hz-Monitor, um Screen tearing zu vermeiden?

Frage 7: Neues Bild darstellen – Screen Tearing vermeiden?

- 24 Bit entsprechen 3 Bytes ($24 / 8 = 3$)
- Für den gesamten Bildschirm braucht man also
 - $1920 * 1080 * 3 = 6.220.800$ Bytes, also 6 MiB pro Puffer
- Es dauert 2 ns ein Byte zu kopieren ($2 * 10^{-9}$)
 - $6.220.800 * 2 * 10^{-9} = \text{ca. } 0,012 \text{ Sekunden} = 12 \text{ ms}$
 - Pro Sekunden können also ca. 80 Kopien gemacht werden
- Um Screen Tearing zu vermeiden kann man einen zweiten Puffer verwenden
 - ein Puffer enthält das aktuelle Bild
 - zur gleichen Zeit wird der andere Puffer mit dem neuen Bild gefüllt
 - Wenn das neue Bild fertig kopiert ist, werden die Puffer getauscht

Aufgabe 1:

Siehe Ende des Dokuments

Aufgabe 2:

Angenommen unser Network Interface Controller (NIC) hat 5 verschiedene Puffer. Der Index vom aktuell verwendeten Puffer befindet sich im Register `nic_buffer_position_reg`. Gegeben ist nun folgende ISR:

```
1  index = *nic_buffer_position_reg;
2  // buffer is a local array
3  copy_buffer_from_driver(&kbuf[index], &buffer);
4  schedule_upper_half(&buffer);
5  *nic_buffer_position_reg++;
6  ack_interrupt();
7  return_from_interrupt();
```

Welches Ziel von Softwaredesign wird in dieser ISR verletzt?
Wie könnte man das beheben?

Aufgabe 2: Welches Designziel wird bei der ISR verletzt?

- ISR verletzt Konsistenz und Datenintegrität
 - Falls ein anderer Interrupt während der Ausführung der ISR auftritt, bevor die Position des Buffers inkrementiert wird, könnte unser Betriebssystem das gleiche Paket zweimal kopieren
- Lösung: Synchronisationsmechanismen
 - Man könnte Interrupts während der ISR deaktivieren oder „maskieren“

```
1  disable_interrupts()
2  index = *nic_buffer_position_reg;
3  // buffer is a local array
4  buffer = copy_buffer_from_driver(&kbuf[index]);
5  schedule_upper_half(&buffer);
6  *nic_buffer_position_reg++;
7  enable_interrupts();
8  ack_interrupt();
9  return_from_interrupt();
```

Aufgabe 3:

Wir benutzen einen Raspberry Pi, um die Wettervorhersage zu laden und auf einem kleinen LCD-Screen darzustellen.

Mit welcher Methode sollte man mit dem LCD-Controller interagieren?

- Programmed I/O (Polling)
 - Interrupt Driven I/O
 - DMA
-
- Sollte das die einzige Anwendung sein, kann man hier sehr gut Polling benutzen
 - Die anderen Methoden kann man auch benutzen, wären aber für so eine Aufgabe unnötig komplex

Aufgabe 3:

Wir benutzen einen Raspberry Pi, um die Wettervorhersage zu laden und auf einem kleinen LCD-Screen darzustellen.

Wir wollen dem Raspberry Pi einen Sensor hinzufügen. Die aktuelle Temperatur und Luftfeuchtigkeit stehen dabei in den Registern des Sensors.

Wir wollen nun die Werte des Sensors alle 5 Minuten lesen. Wie kann man das implementieren?

- Wir können einen Timer implementieren, der alle 5 Minuten die Register des Sensors ausliest

Aufgabe 3:

Wir benutzen einen Raspberry Pi, um die Wettervorhersage zu laden und auf einem kleinen LCD-Screen darzustellen.

Wir wollen dem Raspberry Pi einen Sensor hinzufügen. Die aktuelle Temperatur und Luftfeuchtigkeit stehen dabei in den Registern des Sensors.

Gibt es eine Methode, so dass wir nur die Werte lesen, wenn sie sich auch tatsächlich ändern?

- Ja, wenn wir Interrupt Driven I/O benutzen
- Der Sensor informiert unsere CPU per Interrupt, wenn sich Werte ändern
- Dadurch werden unnötige Lesevorgänge gespart

Aufgabe 3:

Wir benutzen einen Raspberry Pi, um die Wettervorhersage zu laden und auf einem kleinen LCD-Screen darzustellen.

Wir wollen dem Raspberry Pi einen Sensor hinzufügen. Die aktuelle Temperatur und Luftfeuchtigkeit stehen dabei in den Registern des Sensors.

Angenommen unser Board wäre nun batteriebetrieben, wie sollten wir das berücksichtigen?

- Programmed I/O sollte man hier jetzt vermeiden, da es durch die konstanten Überprüfungen mehr Energie verbraucht
- Stattdessen sollte man Interrupt Driven I/O benutzen, da unsere CPU dann die meiste Zeit ungenutzt ist und so Batterie spart

Aufgabe 4:

Eine Soundkarte arbeitet meistens mit 48 kHz und speichert jedes Sample in 16 Bits. Der Soundkarten-Controller benutzt 2 Buffer.

Der erste Buffer wird benutzt, um Sounds auf der Soundkarte abzuspielen.

Der zweite Buffer wird vom Treiber gefüllt. Die Treiber benutzen DMA, um Audiodaten vom Kernel zum Audio Controller zu kopieren.

Welche Puffergröße sollten wir verwenden, wenn wir je 10 ms lange Audiodaten abspielen wollen?

- Die Sample Rate ist 48.000 Samples pro Sekunde
- Jedes Sample ist 2 Bytes (16 Bit) groß
 - Es werden also 96 KB (= 98.304 Bytes/s) pro Sekunde verarbeitet
- $98.304 / 100 = \text{ca. } 983 \text{ Bytes pro } 10 \text{ ms}$
- Unser Puffer sollte also ca. 1 KB groß sein

Aufgabe 4:

Eine Soundkarte arbeitet meistens mit 48 kHz und speichert jedes Sample in 16 Bits. Der Soundkarten-Controller benutzt 2 Buffer.

Der erste Buffer wird benutzt, um Sounds auf der Soundkarte abzuspielen.

Der zweite Buffer wird vom Treiber gefüllt. Die Treiber benutzen DMA, um Audiodaten vom Kernel zum Audio Controller zu kopieren.

Welche Puffergröße sollten wir verwenden, wenn wir je 10 ms lange Audiodaten abspielen wollen?

- Die Sample Rate ist 48.000 Samples pro Sekunde
- Jedes Sample ist 2 Bytes (16 Bit) groß
 - Es werden also 96 KB (= 98.304 Bytes/s) pro Sekunde verarbeitet
- $98.304 / 10 = \text{ca. } 9831$ Bytes pro 100 ms
- Unser Puffer sollte also ca. 10 KB groß sein

Aufgabe 4:

Eine Soundkarte arbeitet meistens mit 48 kHz und speichert jedes Sample in 16 Bits. Der Soundkarten-Controller benutzt 2 Buffer.

Der erste Buffer wird benutzt, um Sounds auf der Soundkarte abzuspielen.

Der zweite Buffer wird vom Treiber gefüllt. Die Treiber benutzen DMA, um Audiodaten vom Kernel zum Audio Controller zu kopieren.

Was ist der Unterschied bei der Latenz und CPU-Auslastung bei den beiden Versionen?

- Je größer der Puffer, desto weniger CPU-Auslastung, da die CPU weniger oft Daten senden muss
- Dadurch entfällt der Overhead, der beim Kopieren von Daten entsteht.
- Latenz erhöht sich, weil wir länger warten müssen, bevor wir wieder kopieren

Aufgabe 4:

Eine Soundkarte arbeitet meistens mit 48 kHz und speichert jedes Sample in 16 Bits. Der Soundkarten-Controller benutzt 2 Buffer.

Der erste Buffer wird benutzt, um Sounds auf der Soundkarte abzuspielen.

Der zweite Buffer wird vom Treiber gefüllt. Die Treiber benutzen DMA, um Audiodaten vom Kernel zum Audio Controller zu kopieren.

Was ist der Vorteil von den 2 Puffern, die wir benutzen?

- Würden wir nur einen Puffer benutzen, müssten wir nach Ende des Soundschnipsels warten, weil der neue Soundschnipsel erst nach Ende der Wiedergabe geladen werden kann
- Mit dem zweiten Puffer können wir schon während der Wiedergabe des ersten Schnipsels den zweiten laden.
- Mit nur einem Puffer könnte die Audiowiedergabe immer wieder unterbrochen werden

Aufgabe 4:

Eine Soundkarte arbeitet meistens mit 48 kHz und speichert jedes Sample in 16 Bits. Der Soundkarten-Controller benutzt 2 Buffer.

Der erste Buffer wird benutzt, um Sounds auf der Soundkarte abzuspielen.

Der zweite Buffer wird vom Treiber gefüllt. Die Treiber benutzen DMA, um Audiodaten vom Kernel zum Audio Controller zu kopieren.

Wie beeinflusst DMA die Effizienz vom Audiodatentransfer im Vergleich zu CPU-driven data transfer?

- Mit DMA können die Audiodaten direkt vom Speicher in den Audio Controller geladen werden, ohne dass die CPU involviert ist.
- So kann die CPU in der Zeit andere Operationen ausführen und der Overhead des Datentransfers wird reduziert

Task 1 - FCFS

Freitag, 14. Juni 2024 12:00

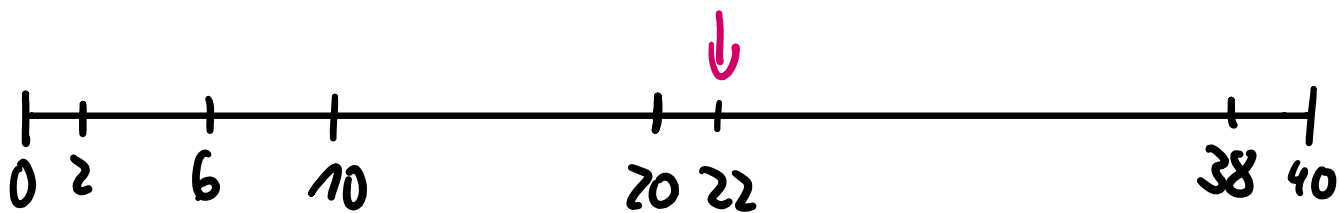
Task 1

Disk requests come in to the disk driver for cylinders 10, 22, 20, 2, 40, 6, and 38, in that order. A seek takes 6 msec per cylinder. How much seek time is needed for

1. First-Come, First-Served (FCFS).
2. Shortest Seek First (SSF).
3. Elevator algorithm (SCAN, initially moving upward).

In all cases, the arm is initially at cylinder 20. There are 40 cylinders in total.

1. FCFS:



$$\begin{array}{lll} 20 \rightarrow 10 & = & 10 \\ 10 \rightarrow 22 & = & 12 \\ 22 \rightarrow 20 & = & 2 \\ 20 \rightarrow 2 & = & 18 \\ 2 \rightarrow 40 & = & 38 \\ 40 \rightarrow 6 & = & 34 \\ 6 \rightarrow 38 & = & 32 \end{array}$$

$$\overline{146} * 6 = 876 \text{ ms}$$

Task 1 - SSF

Freitag, 14. Juni 2024 12:00

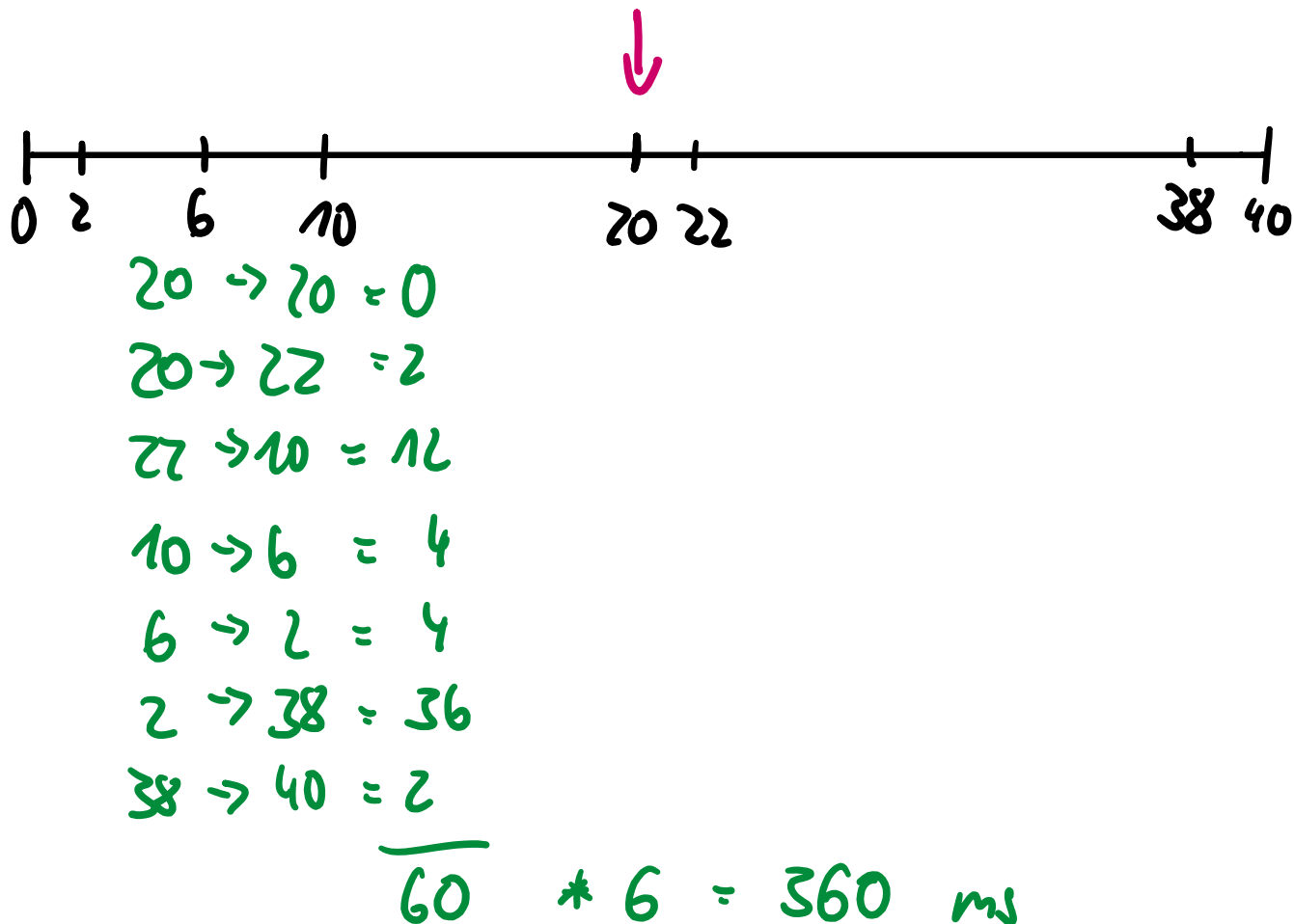
Task 1

Disk requests come in to the disk driver for cylinders 10, 22, 20, 2, 40, 6, and 38, in that order. A seek takes 6 msec per cylinder. How much seek time is needed for

1. First-Come, First-Served (FCFS).
2. Shortest Seek First (SSF).
3. Elevator algorithm (SCAN, initially moving upward).

In all cases, the arm is initially at cylinder 20. There are 40 cylinders in total.

2. SSF :



Task 1 - SCAN

Freitag, 14. Juni 2024 12:00

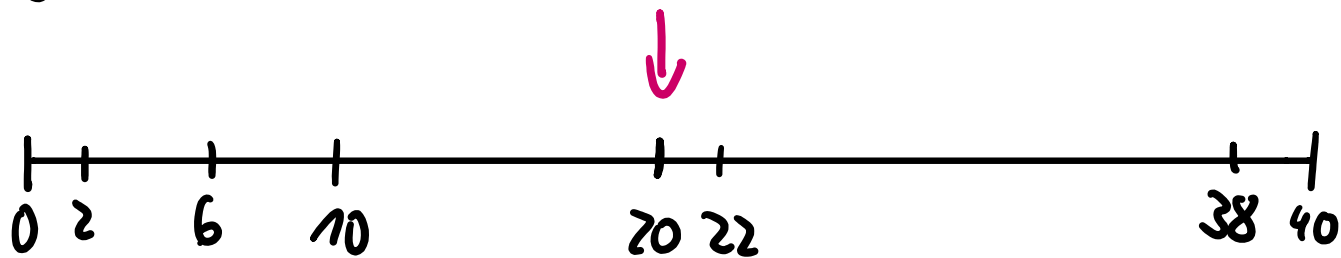
Task 1

Disk requests come in to the disk driver for cylinders 10, 22, 20, 2, 40, 6, and 38, in that order. A seek takes 6 msec per cylinder. How much seek time is needed for

1. First-Come, First-Served (FCFS).
2. Shortest Seek First (SSF).
3. Elevator algorithm (SCAN, initially moving upward).

In all cases, the arm is initially at cylinder 20. There are 40 cylinders in total.

3. SCAN :



$$20 \rightarrow 20 = 0$$

$$20 \rightarrow 22 = 2$$

$$22 \rightarrow 38 = 16$$

$$38 \rightarrow 40 = 2$$

$$40 \rightarrow 10 = 30$$

$$10 \rightarrow 6 = 4$$

$$6 \rightarrow 2 = 4$$

$$\underline{58}$$

$$* 6 = 348 \text{ ms}$$